设计模式期末复习

- 设计模式概述
- 面向对象概述
    - 单一职责原则
    - 开闭原则
    - 里氏代换原则
    - 依赖倒置原则
    - 组合复用原则
    - 接口隔离原则
    - 迪米特法则
- 设计模式
    - 行为型模式
        - 工厂模式
            - 简单工厂
            - 工厂方法
            - 抽象工厂
        - 建造者模式
        - 原型模式
    - 结构型模式
        - 适配器模式
        - 桥接模式
        - 组合模式
        - 装饰器模式
        - 外观模式
        - ~~享元模式~~
        - 代理模式
    - 行为型模式
        - ~~责任链模式~~
        - 命令模式
        - ~~解释器模式~~
        - 迭代器模式
        - ~~中介者模式~~
        - ~~备忘录模式~~
        - 观察者模式
        - 状态模式
        - 策略模式
        - ~~模板方法模式~~
        - ~~访问者模式~~

设计模式概述

模式是在特定环境下人们解决某类重复出现问题的一套成功或有效的解决方案。

设计模式是在**特定环境**下为解决某一**通用软件设计问题**提供的一套**定制的解决方案**，该方案描述了**对象和类之间的相互作用**。

设计模式是一套被反复使用、多数人知晓的、经过分类别的代码设计经验总结

目的：为了可**重用**代码、让代码更容易被他人**理解**、提高代码**可靠**性

设计模式的**组成**：模式名称、问题、解决方案、效果

分类：

- 根据目的分类：创建型模式（创建对象）、结构性模式（处理类和对象的组合）、行为型模式（描述类和对象如何交互和怎么分配职责）
- 根据范围分类：类模式（处理类和子类之间的关系）、对象模式（处理对象之间的关系）

范围\目的	创建型模式	结构型模式	行为型模式
类模式	工厂方法模式	（类）适配器模式	~~解释器模式~~ ~~模板方法模式~~
对象模式	抽象工厂模式 建造者模式 原型模式 单例模式	（对象）适配器模式 桥接模式 组合模式 装饰模式 外观模式 ~~享元模式~~ 代理模式	~~职责链模式~~ 命令模式 迭代器模式 ~~中介者模式~~ ~~备忘录模式~~ 观察者模式 状态模式 策略模式 ~~访问者模式~~

面向对象概述

面向对象**优点**：可维护、可复用、可扩展、灵活性高

面向对象编程**特性**：封装、继承、多态

衡量软件质量的属性：

- 可维护性：指软件能够被理解、改正、适应及扩展的难易程度
- 可复用性：指软件能够被重复使用的难易程度

目标：

- 实现设计方案或者源代码的**复用**
- 确保系统易于**扩展和修改**
- 具有良好的**可维护性**

面向对象设计原则：

- 单一职责原则

  一个对象应该只包含单一的职责，并且该职责被完整地封装在一个类中
- 开闭原则（总原则）

  软件实体应当对扩展开放，对修改关闭
- 里氏代换原则

  任何父类出现的地方，子类都可以替代出现
- 依赖倒转原则

  要依赖于抽象，不要依赖于具体的实现。抽象不应该依赖于细节，细节应该依赖于抽象
- 接口隔离原则【使用频率最少】

  客户端不应该依赖那些它不需要的接口

- 合成复用原则

  优先使用对象组合，而不是继承来达到复用的目的

- 迪米特法则

  又称最少知识原则，一个对象应当对其他对象尽可能少的了解，不和陌生人说话

开合 单隔 倒换迪米特

单一职责原则

描述：

- 一个对象应该**只包含单一的职责**，并且该职责被完整地封装在一个类中。
- 就一个类而言，应该**仅有一个引起它变化的原因**

分析：

- 一个类职责越多，复用的可能性就越小
- 当一个职责变化时，可能导致对象状态变化，从而影响其他职责的工作。因此，应该将不同的职责封装到不同的类中
- 单一职责原则是实现高内聚，低耦合的指导方针

好处：

- 类的复杂性降低，职责明确
- 可读性、可维护性高

使用场景：接口设计、类的设计、方法的设计

开闭原则（总原则）

描述：

- **软件实体**应当对**扩展开放，对修改关闭。**
- 即开放-封闭原则，软件实体（**类、模块、函数等**）应该可以扩展，但是不能修改。

分析：

- 软件实体可以是一个**软件模块**、一个由**多个类组成的局部结构**或一个**独立的类**
- 开闭原则是指软件实体应尽量在不修改原有代码的情况下进行扩展，面对需求，对程序的改动是**增加**新代码，不是**更改**现有代码。
- 稳定的抽象层，灵活的实现层
- 找到系统可变的因素，将其封装起来

里氏代换原则

继承优点：

- 代码共享，可以减少类的工作量
- 提高代码复用
- 子类可以对父类的方法进行重写
- 提高代码的可扩展性

继承缺点：

- 继承是入侵式的
- 降低代码的灵活性
- 增强了耦合性

- Java单一继承，C++多重继承

描述：

- 如果对每一个类型为S的对象o1，都有类型为T的对象o2，使得以T定义的所有程序P在所有的对象o1都代换成o2时，程序P的行为没有发生变化，那么类型S是类型T的子类型。
- 所有引用**基类**的地方必须能透明地使用其**子类**的对象

分析：

- 将一个基类对象替换成它的子类对象，程序将不会产生任何错误和异常
- 尽量使用**基类类型**来对对象进行定义
- 运用时，父类应该是**抽象类或者接口**，让子类继承父类或者实现父类
- 无需修改原有类型的代码。易于通过新增的方式来扩展系统功能

依赖倒置原则（面向接口编程）

描述：

- 高层模块不应该依赖低层模块，它们都应该依赖抽象。抽象不应该依赖于细节，细节应该依赖于抽象。
- 面向接口编程

分析：

- 尽量在传递参数或者关联关系中，传递层次高的抽象类或者接口
- 编程的过程中，尽量使用抽象层进行编程
- 针对抽象层编程，具体类通过**依赖注入**的方式注入到其他对象。构造注入、设值注入（Setter）、接口注入
- 面向对象设计的主要机制

接口隔离原则

描述：

- 客户端不应该依赖那些它不需要的接口。
- 类间的依赖关系应该建立在最小的接口上。

分析：

- 接口细化，如果接口内容过于庞杂，需要将其划分为更小的
- 每一个接口承担相对独立的角色
- 接口隔离 VS 单一职责
    - 单一职责更加重视类的实现上的单一，面向开发者，注重业务逻辑的职责划分
    - 接口隔离更多面向使用者，隔离要求接口方法尽可能精炼

接口隔离原则：

- 接口要尽量小，不臃肿
- 接口要高内聚，减少对外的交互
- 定制服务。为每一类（某个接口）的使用者提供**单独的服务**
- 接口的设计是有限度的。细粒度也并不是越小越好

合成复用原则

也成为，"组合\聚合原则"。

描述：

- 优先使用对象组合，而不是通过继承来达到复用的目的

分析：

- 合成复用原则就是在一个新对象里面通过关联关系或者组合关系来使用一些对象，使之成为新对象的一部分，达到复用的目的
- 新对象**通过委派调用已有对象的方法**达到复用功能的目的
- 复用时要**尽量使用组合/聚合关系（关联关系），少用继承**
- 继承复用 VS 组合/聚合复用
    - **继承复用**：实现简单，易于扩展。破坏系统的封装性；继承来的实现是静态的，不可能在运行时改变；没有足够的灵活性
    - **组合/聚合复用**：耦合度相对较低，有选择性的调用成员对象的操作。

Coda法则;

- 子类是基类的一个特殊种类（is-a），而不是基类的一个角色（has-a）
- 永远不会出现永远不会出现将子类替换成另一个类的子类的情况
- 子类具有扩展基类的责任，而不是去置换（Override）
- 只有在分类学上有意义时，才可以使用继承

迪米特法则

描述：

- 每一个软件单位对其他的单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。

分析：

- 每一个软件实体应该尽可能少的与其他实体发生相互作用
- 一个对象应该对其他对象应该有**最少的了解**（接口隔离）
- 一个类对其他自己需要耦合或者调用的类应该**知道的尽量少**
- 应用迪米特法则可以**降低系统的耦合度**

典型应用：**外观模式**

创建型模式

- 将对象的创建过程和使用过程进行分离
- 创建型模式更加关注对象的创建过程
- 创建型模式，更加符合单一职责，外界只需要知道他们的共同接口即可

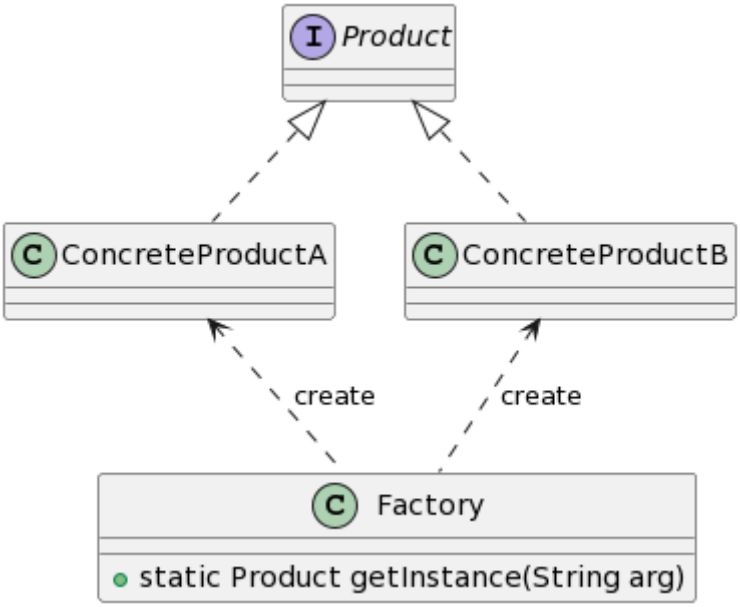
简单工厂模式

别名：静态工厂方法模式

问题：只需要某个参数，就可以获得相应的对象，这个工厂方法定义为 `static` 类型。

定义：根据不同的参数获取不同类的实例，这些不同的类通常都有相同的父类。

类图：



优点：

- 只关心**创建过程**，不关心具体业务，符合单一职责
- 增加新的产品只需要，新增具体的产品并且修改工厂方法
- **简单**，只需要简单调用类的静态方法，传入参数即可
- 降低了系统的**耦合度**，增加**灵活性。**实际开发中，传入的参数可以在配置文件中，进一步降低了系统耦合度

缺点：

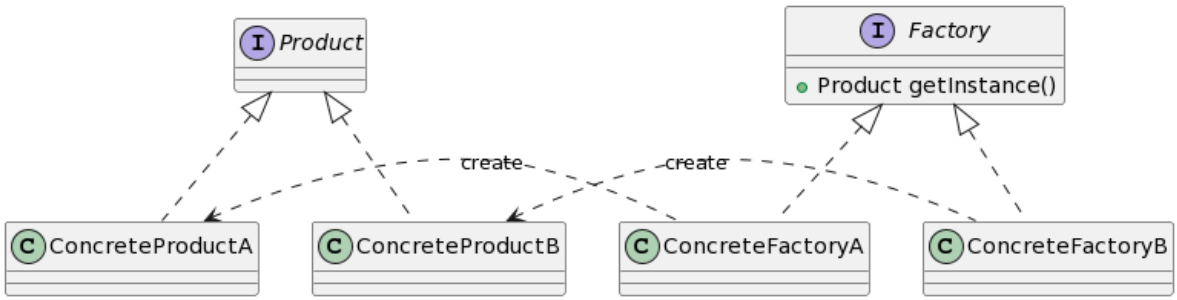
- 工厂类承担了所有产品的创建逻辑，**职责过重**
- **增加了系统中类的个数**，增加了系统的复杂度
- 系统扩展困难，违背**开闭原则**

工厂方法模式

别名：虚拟构造器，多态工厂

定义：工厂父类负责定义创建产品对象的**公共接口**，类的实例化操作延迟到子类。

类图：



优点：

- 向客户**隐藏具体产品类实例化**的具体细节
- 让工厂自主确定实例化何种产品对象
- 完全符合**开闭原则**，可以在不修改工厂角色的前提下引进新的产品

缺点：

- 复杂度增加，类的个数随着产品类型的增加，**成对增加**
- 增加系统的理解难度

抽象工厂模式

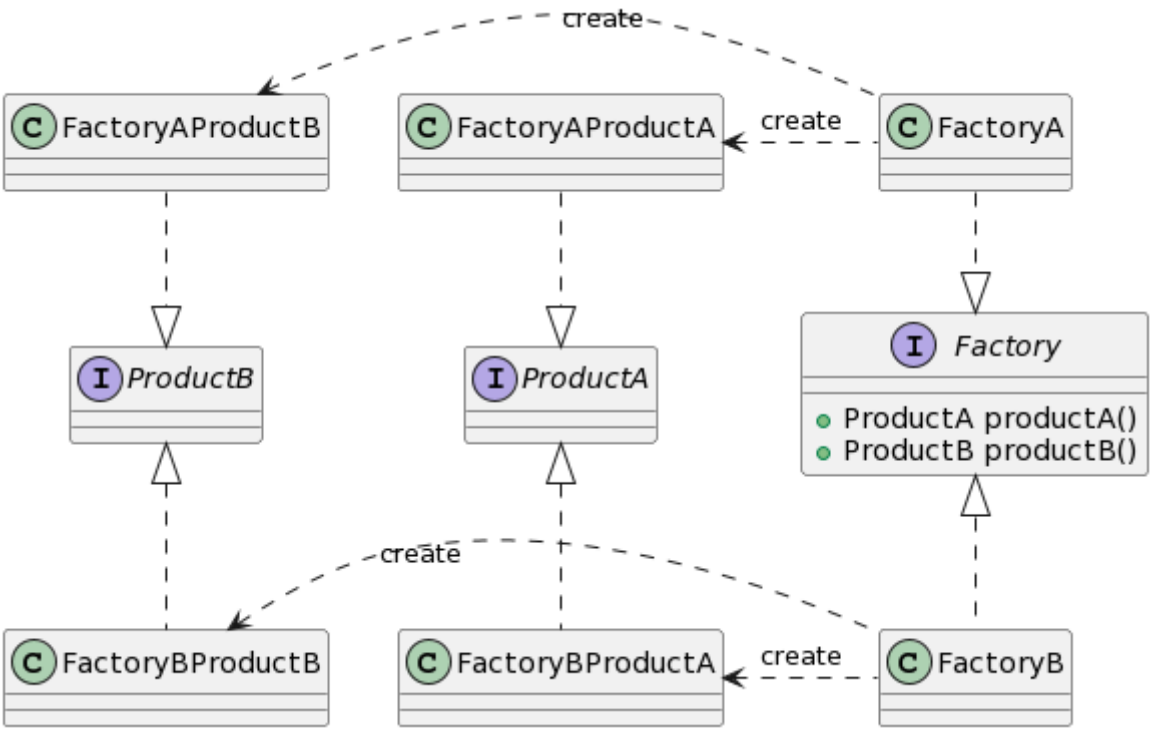
别名：Kit模式

概念：

- 产品等级结构：产品的继承结构
- 产品组：同一个工厂生产的不同产品等级结构的一组产品

定义：提供一个创建一系列相关或者依赖对象的接口，而无需指定它具体的类。

类图：



优点：

- **隔离**了具体类的生成
- 保证客户可以**始终使用同一个产品组**的对象
- **符合开闭原则。**在产品组一侧符合开闭原则，只需要简单增加工厂即可

缺点：

- 增加新的产品等级结构比较复杂，**违背开闭原则**

使用条件：

- 不依赖于产品实例如何被创建，组合和表达的细节
- 每次只使用某一产品组
- 属于同一产品组的产品将在一起使用
- 产品等级结构稳定

三种工厂模式对比

三个模式都属于**工厂模式**，含义就是使用**工厂**生产**产品。**

将生产产品的一个或者一系列方法封装到一个类中，这样的类叫做**工厂类**；被实例化的类那就是**产品类。**

1. 简单工厂

优点：简单，静态工厂方法，可以创建所有类型的产品

缺点：职责重，单个类复杂度高；不符合开闭原则

描述：支持**同一等级结构**中的任意产品，**不支持**扩展增加产品

2. 工厂方法

   优点：降低工厂的负担；完全符合开闭原则

   缺点：类的个数成对增加；对于形成产品族的情况处理复杂

   描述：支持**同一等级结构**的固定产品（**支持**扩展增加产品）

3. 抽象工厂

   优点：针对产品族场景的优化；产品组符合开闭原则

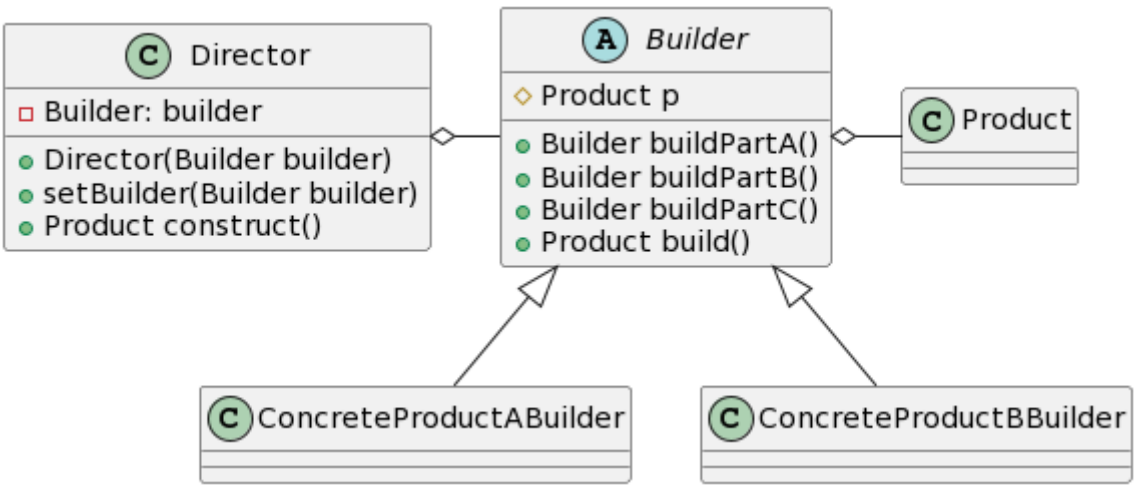
   缺点：模式复杂，只适用于产品族的场景

   描述：支持不同产品族的全部产品（**不支持**增加产品类型，**支持**增加产品族） 抽象工厂着重于单个的产品对象 错 **多个产品**

建造者模式

定义：将一个复杂对象的**构建**，**与**它的**表示分离**，使得同样的构建过程创建不同的表示。建造者模式就是一步一步构建一个复杂的对象，它允许用户只通过指定的类型和内容就可以构建他们，用户不需要知道内部的具体构建细节。

类图：



优点：

- 将产品的**创建过程和产品的表示解耦**，使得创建过程可以创建不同的产品对象
- 可以很方便的创建具体的建造者或者增加新的具体建造者，符合**开闭原则**
- 便于更加**精细的控制**产品的创建过程

缺点：

- 产品之间差异很大，不适合使用建造者模式
- 如果产品内部变化复杂，可能会需要定义很多具体建造这类来适应这种变化

场景：

- 需要生成的对象有很复杂的内部结构
- 需要生成的对象属性存在相互依赖
- 对象的创建过程独立于创建该对象的类
- 隔离复杂对象的创建和表示

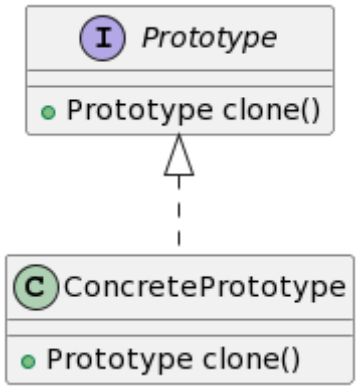
原型模式

定义：用原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象

工作原理：

- 通过请求原型对象复制自己来实现创建的过程
- 工厂类就是原型自身，工厂方法是负责对象的**克隆方法**
- 克隆创建的对象是**全新的对象**，有独立的内存空间
- 原型模式可以得到一系列相似但是不完全相同的对象

类图：



优点：

- **简化对象的创建过程**，提高新实例的创建效率
- **扩展性**比较好
- **简化创建结构**，简单的一个方法可以克隆出一的新的对象
- 可以使用深拷贝的方式来保存**对象的状态**

缺点：

- 需要为每一个类实现克隆方法，对已有的类进行改造时，需要修改源代码，**违背了开闭原则**
- **深拷贝**的方式需要编写**较为复杂**的代码

场景：

- **创建新对象成本比较大**，可以通过复制对象来获得
- 需要**保存**对象的状态，而且对象的**状态变化小**
- 避免使用分层次的工厂类来创建分层次的对象

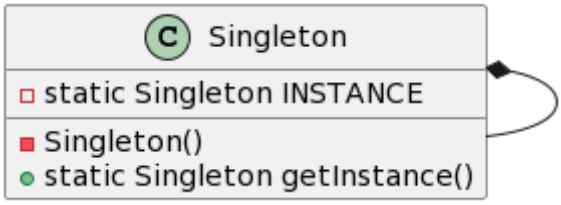
扩展：

- 原型管理器

单例模式

定义：确保一个类只有一个实例，而且自行实例化并且向整个系统提供这个实例。

类图：



优点：

- 对唯一实例受控访问
- 节约资源，提高系统性能
- 允许可变数量（有限个）的实例（多例模式）

缺点：

- 扩展困难（缺少抽象层）
- 单例类职责重。一个业务类不应该关心自己是不是单例，但是单例模式要把模式和业务混合在一起
- 垃圾回收机制，可能导致共享单例对象的状态丢失

场景：

- 系统**只需要一个这个类的实例**，或者资源**只允许创建一个对象实例**
- 客户调用类的单个实例，**只允许使用一个公共访问点**

结构型模式

- 将不同的类和对象组合在一起
- 结构性模式主要分为：**类结构型模式**和**对象结构型模式**
- 类结构模式：一般只存在**继承**和**实现**关系
- 对象结构模式：通过**关联**关系，在一个类的实例中使用另一个类的实例
- 为了遵循**合成复用原则**尽量使用关联关系代替继承关系，因此大部分结构型模式都是**对象结构型模式**

适配器模式

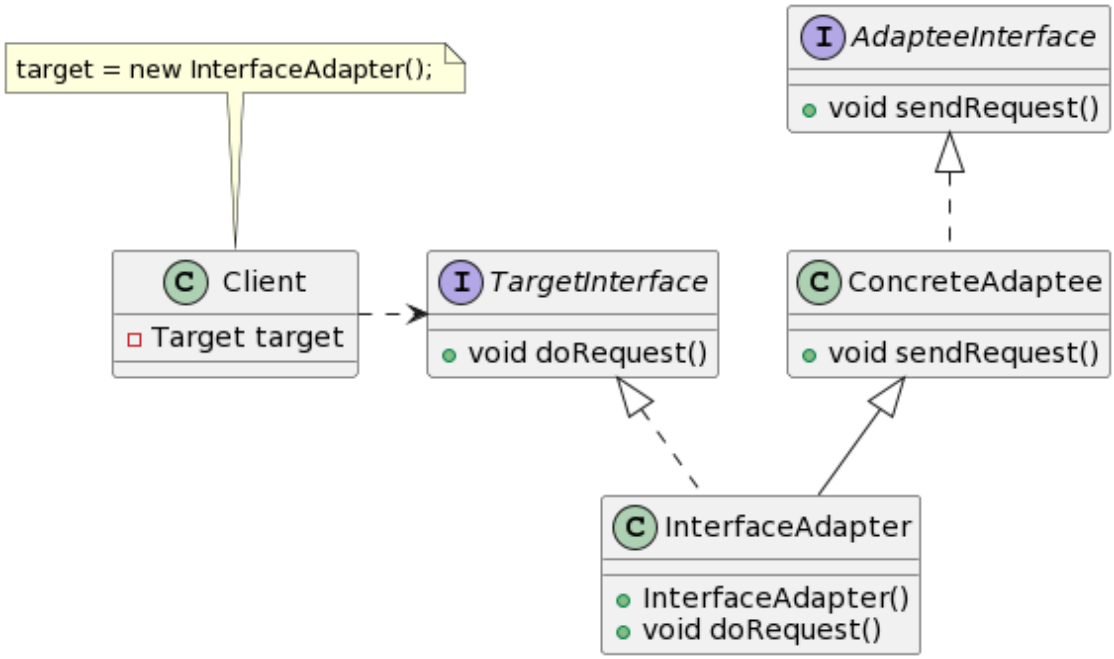
别名：包装器（Wrapper）模式

定义：将**一个接口**装换成客户希望的**另一个接口**使得接口不兼容的类可以一起工作

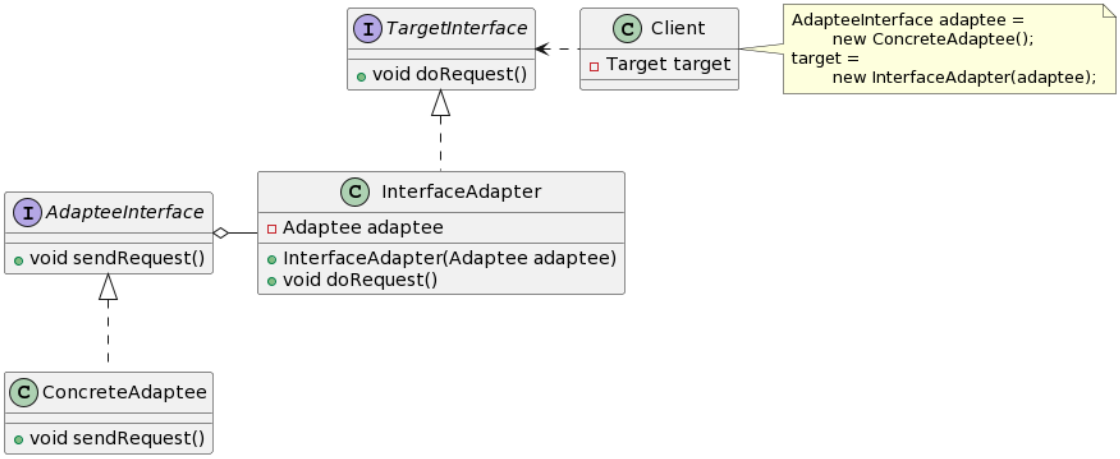
描述：适配器模式可以看作是现有系统进行不久以及对现有类进行重用的模式。或者是第三方接口可以满足本系统的需求，但是接口并不符合本系统的接口规范，这个时候就需要使用适配器模式对第三方接口进行适配。

类图：

- 类适配器



- 对象适配器



优点：

- **目标类和适配者的解耦**，通过引入一个适配器重用现有功能代码
- 增加了类的**透明性和复用性**，提高了适配者的复用性
- **灵活性和扩展性非常好**，更换适配器，符合开闭原则
- 类适配器：由于继承关系，置换一些适配者的方法会很方便
- 对象适配器：可以适配存在继承层次的适配者，或者把多个不同的适配者适配到同一个目标

缺点：

- 类适配器模式（继承复用）
  - **一次最多适配一个适配者类**，不能同时适配多个适配者（多继承也不过是隔靴搔痒）
  - 适配者不能是最终类（final）
  - **目标抽象类只能为接口，不能为类**（有多**继承**机制就行）
- 对象适配器模式（合成聚合复用）
  - 在适配器中置换适配者类的某些方法比较麻烦

场景：

- 需要使用一些现有的类，但是这些类的接口符合系统需要
- 创建一个可复用的类

扩展：

- 缺省适配器模式（Default Adapter Pattern）

  不需要实现要给接口所提供的所有方法时，可以设计一个抽象类实现这个接口
- 双向适配器模式

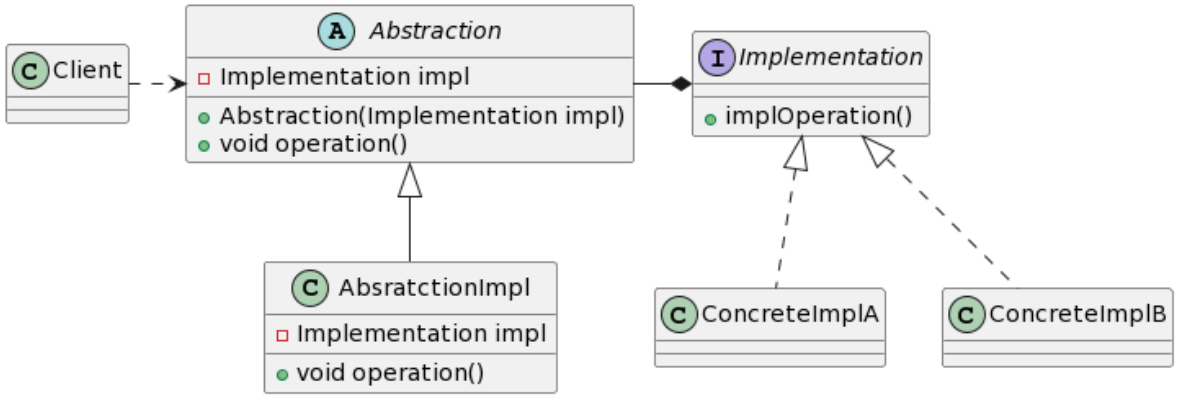
桥接模式

别名：柄体(Handle and Body)模式或接口(Interface)模式

描述：**使用抽象关联取代了传统的多继承，将类之间的静态继承关系转换为动态的对象组合关系**

定义：将抽象部分与它的实现部分分离，使得他们都有独立的继承体系。

类图：



优点：

- **脱耦**：将强关联关系转换弱关联关系，符合合成复用原则
- 分离抽象接口极其实现部分
- 取代继承的方案，使得Java也能有更加灵活的多继承方案，并且**极大减少了子类的个数**
- **提高了系统的可扩展性**，符合**开闭原则**

缺点：

- **增加系统的理解和设计难度**，由于关联关系建立在抽象层，要求开发者一开始就要对抽象层进行设计和编程
- **正确识别出独立变化的维度并不是一件容易的事情**

场景：

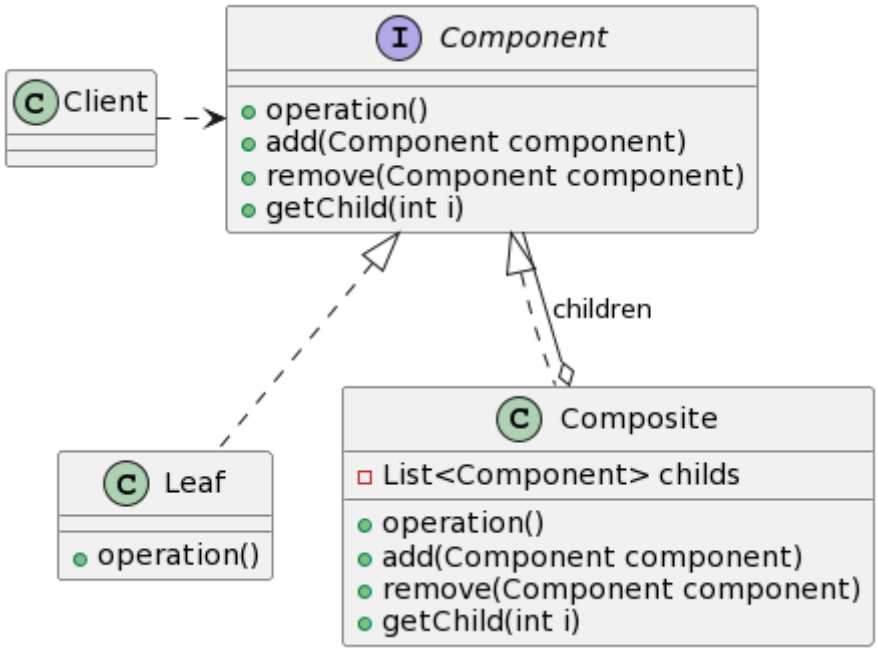
- 需要在抽象和实现层面增加更多的灵活性，**避免两个层次之间存在静态的继承关系**
- 抽象部分和实现部分可以一继承的方式**独立扩展而互不影响**
- **一个类存在两个独立变化的维度**
- 不希望使用继承或者因为多层继承导致系统类的个数急剧增加

组合模式

描述：组合模式使用面向对象的方式来处理树形结构；使用了递归调用的机制来对整个结构进行处理

定义：组合多个对象形成**树形结构**以表示"部分-整体"的结构层次。组合模式对**单个对象（即叶子对象）**和**组合对象（即容器对象）**的使用具有统一性。

类图：



优点：

- **定义分层次的复杂对象**，表示对象的全部或者部分层次；**让客户端忽略层次之间的差异**，方便对整个层次结构进行控制
- 客户端可以**一致使用一个组合结构或者其中单个对象**，不必关心处理的是单个对象还是组合结构，**简化客户端代码**
- 增加新的容器或者新的构件都很方便，符合开闭原则
- 为**树形结构的面向对象实现**提供了一种灵活方案

缺点：

- 设计更加抽象，理解难度大
- 增加新构件的时候，**很难对容器中的构建类型进行限制**

场景：

- **具有整体和部分的层次结构**中使用，而且客户端希望能够以一种**一致地对待他们**
- 面向对象语言中，需要处理一种树形结构，而且每个阶节点有自己的状态和行为
- 在系统中能够**分离处理叶子对象和容器对象**，而且他们的类型并不确定

装饰器模式

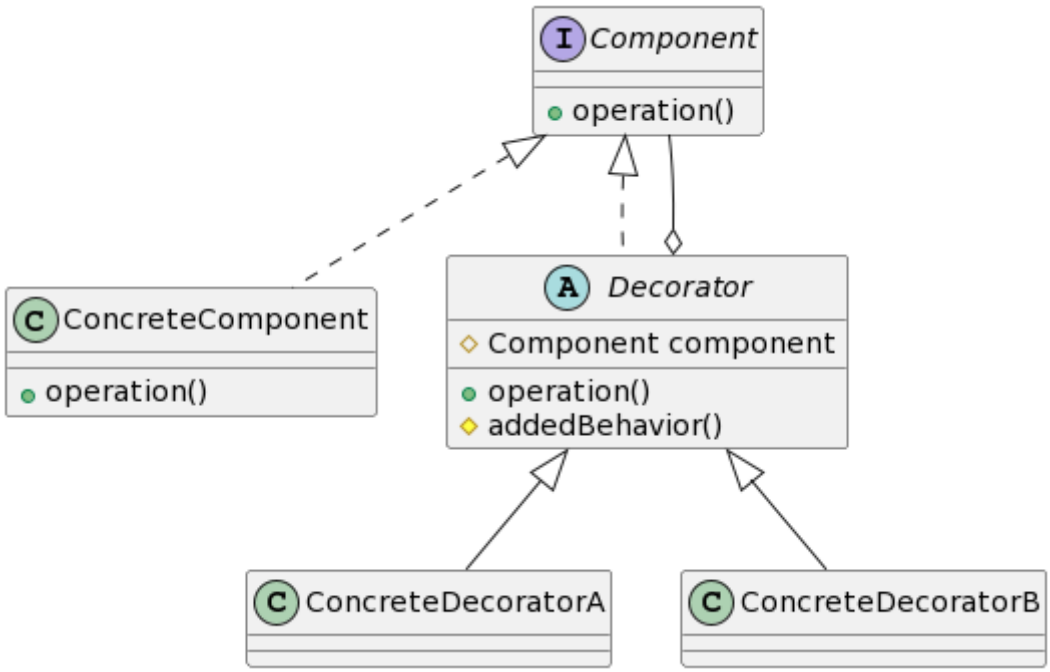
动机：

- 希望在原有对象的功能基础上给新的对象增加一些新的行为
- 用于替代继承的一种技术，给对象动态增加职责，使用关联关系取代对象之间的继承关系
- 在装饰器方法中，既可以调用被修饰对象的方法，也可以在这个基础上增加新的方法，扩展原有功能

定义：**动态地给对象增加一些额外的职责**，就增加对象来说，装饰器模式比生成子类实现更为灵活

- 对客户以透明的方式，给一个对象附加更多的责任
- 可以在不需要创建新的子类的情况下，让对象的功能扩展

类图：



分析：

透明装饰器模式

- 要求客户端完全**针对抽象编程**，不应该将对象声明为具体的类型或者装饰器类型，应该全部声明为抽象的构件类型
- 对于客户端而言，具体构建对象和具体装饰器对象没有任何区别。可以对一个**已经被装饰过的对象进行多次装饰**
- 无法在客户端单独调用新增的方法

半透明装饰器模式

- 用**具体装饰类型来定义装饰之后的对象**，而具体构建对象使用抽象构件来声明
- **具体构建类型**无需关心，是**透明**的；但是**具体的装饰类**必须指定，是**不透明**的
- 可以单独调用新增的方法（职责）
- 不能对一个**对象多次装饰**，而且客户端要**区别对待**具体构件对象和装饰后的对象

优点：

- 装饰器模式比继承更加**灵活**，不会导致子类个数急剧飙升
- 装饰器模式，可以**动态扩展一个对象**的功能
- 一个装饰器可以**多次被装饰**（透明装饰）
- **具体构件和具体装饰器可以独立变化**，用户根据需要增加具体的新构件或者具体的装饰器类，符合**开闭原则**

缺点：

- 使用装饰器模式，将产生很多小对象，在一定长度上会影响程序的性能
- 比继承更容易出错，排错也更加困难

场景：

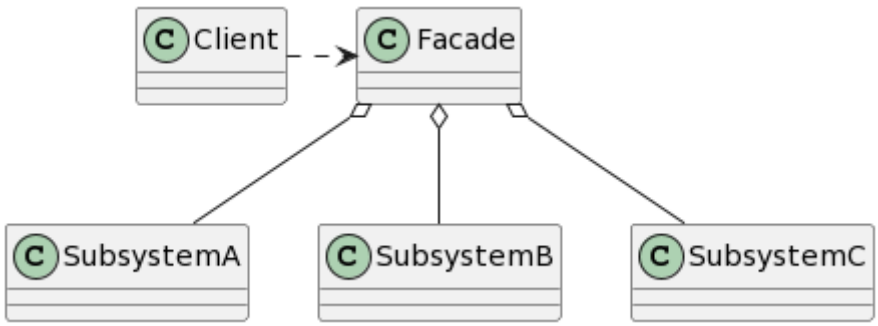
- **以动态透明的方式给单个对象添加职责**
- 不能采用继承的方式对系统进行扩展或者采用继承不利于系统扩展和维护的时候，可以使用装饰模式

外观模式

描述：**为复杂的系统提供统一的入口**

定义：外部与子系统通信**通过一个统一的外观对象进行**，为子系统中的一组接口**提供一个统一的入口。**

类图：



分析：

- **迪米特法则**的具体实现
- 通过引入一个新的外观类**降低原有系统的使用难度**，**降低**客户和子系统的**耦合程度**

优点：

- **减少**了客户端所需要**处理的对象数量**
- 实现了客户端和子系统之间的**松耦合关系**
- 子系统的修改对其他子系统之间没有任何影响，子系统内部的修改影响到外观对象

缺点：

- 不能很好的**限制客户端直接使用子类系统**
- 如果设计不当，**增加新的子系统可能需要修改外观类的源代码**

场景：

- 为一系列复杂的子系统提供**统一的简单入口**
- 减少客户端和子系统之间的**复杂依赖关系**
- **层次化系统结构**中，外观模式可以为每一个层次的系统定义一个入口，不同的系统之间不直接相互联系，而是通过外观类建立联系，降低层次之间的耦合度

模式扩展：

- 一个系统有多个外观类
- 不要试图通过外观类为子系统增加新行为
- 外观模式与迪米特法则
- **抽象外观类的引入**

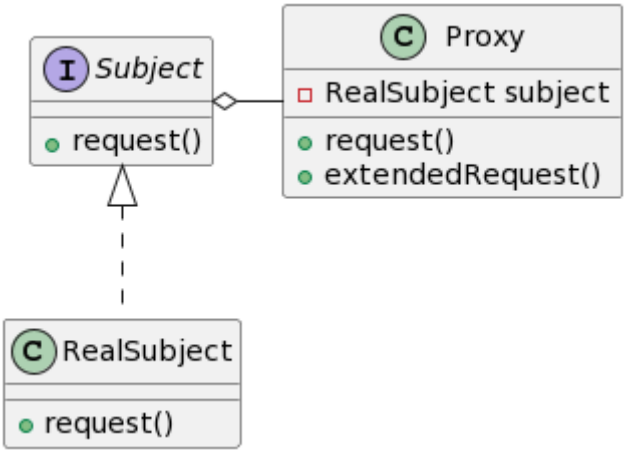
  如果需要增加，删除或者更换与外观类交互的子系统，**必须修改外观类或者客户端的源代码，这违背了开闭原则**，因此可以通过引入**抽象外观类**对系统进行改进，就可以一定程度上解决这个问题

代理模式

动机：引入一个新的对象来实现对真实对象的操作，或者将新的对象作为真实对象的一个替身；**引入一个代理对象来访问一个对象**

定义：给某个对象**提供一个代理**，并**由代理对象控制原对象的引用。**

类图：



几种常见的代理模式：

- 远程代理（Remote Proxy）
- 虚拟代理（Virtual Proxy）
- 保护代理（Protected Proxy）：**不同用户提供不同访问权限**
- 缓冲代理（Cache Proxy）
- 智能引用代理（Smart Reference Proxy）

优点：

- 协调调用者和被调用者，**降低了系统的耦合度**
- 增加和更换代理类无需修改源代码，符合开闭原则

缺点：

- 有些类型的代理模式可能造成请求的处理速度慢（**保护代理**）
- 有些代理模式实现过程可能较为复杂（**远程代理**）

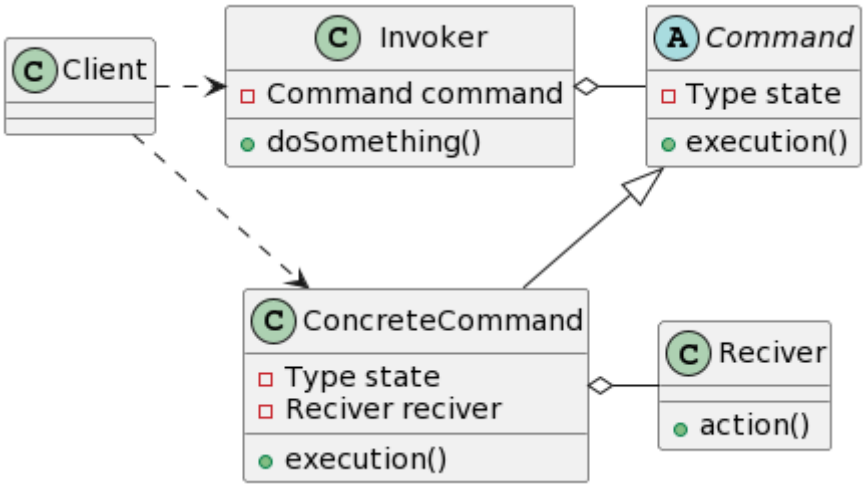
行为型模式

- 行为型模式定义了系统中对象之间的交互与通信，研究系统在运行时的相互通信与写作，进一步明确对象的职责，包括对对象复杂的流程的控制

命令模式

定义：**将一个请求封装为一个对象**，从而使我们**对不同的请求对客户进行参数化**；对**请求排队**或者**记录请求日志**，以及**支持可撤销的操作**

类图：



分析：

- 命令模式是对**请求进行的封装**
- 请求的发送者和接收者，**完全解耦**
- 使得请求的一方**不必知道接受请求一方的接口**，更不必关心**发出请求的具体操作流程**
- 发送者只需要关心如何发出请求，**不关心请求如何完成**
- **一个请求对应一个命令**

优点：

- **降低系统的耦合度**
- 新的命令很容易被添加到新的系统中，符合**开闭原则**
- 比较容易去实现一个**命令队列或者宏命令（组合命令）**
- 为请求的**撤销（Ctrl-Z）和恢复（Ctrl-Y）**提供了一种设计和实现方案

缺点：

- 可能会导致系统有过多的具体命令类

场景：

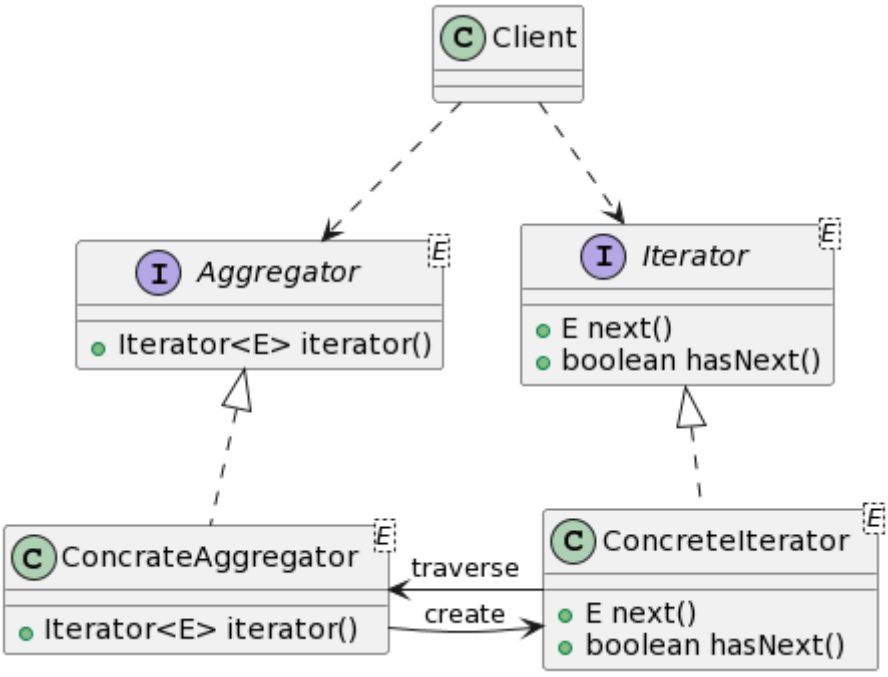
- **将请求者和接收者解耦**
- **在不同时间指定请求、将请求排队和执行请求**
- 需要支持**撤销（Undo）和恢复（Redo）**操作
- 需要将一组命令组合形成**宏命令**

迭代器模式【游标模式】

动机：**迭代器用于对一个聚合对象进行遍历**，将遍历操作从聚合对象中分离出来，**聚合对象只负责存储数据**，而遍历数据由迭代器来完成。访问一个聚合对象中的元素，但是又不需要暴露它的内部结构，那么就需要迭代器模式

定义：提供一种方法来**访问聚合对象**，而**不用暴露这个聚合对象的内部表示**

类图：



分析：

- 聚合对象的职责
  - **存储**数据：最基础的职责
  - **遍历**数据：既可以是变化的，又是可以分离的
- **将遍历数据的行为从聚合对象中分离出来**，封装到迭代器中，简化聚合对象的设计，符合**单一职责原则**

优点：

- 以不同的方式遍历一个聚合对象，在同一个聚合对象上可以定义多种遍历方式
- 简化了聚合对象
- 无论是增加新的聚合类，还是增加新的迭代器都很方便，符合**开闭原则**

缺点：

- 类的个数成对增加，**增加了整个系统的复杂性**
- **抽象迭代器的设计难度大，需要充分考虑到系统将来的扩展。设计一个考虑全面的抽象迭代器并不是一件容易的事情**

情景：

- 访问聚合对象本身但是**不能暴露内部结构**
- 需要提供聚合对象的**多种遍历方式**
- **为遍历提供一个统一的接口，客户端可以一致性操作**

观察者模式

别名：

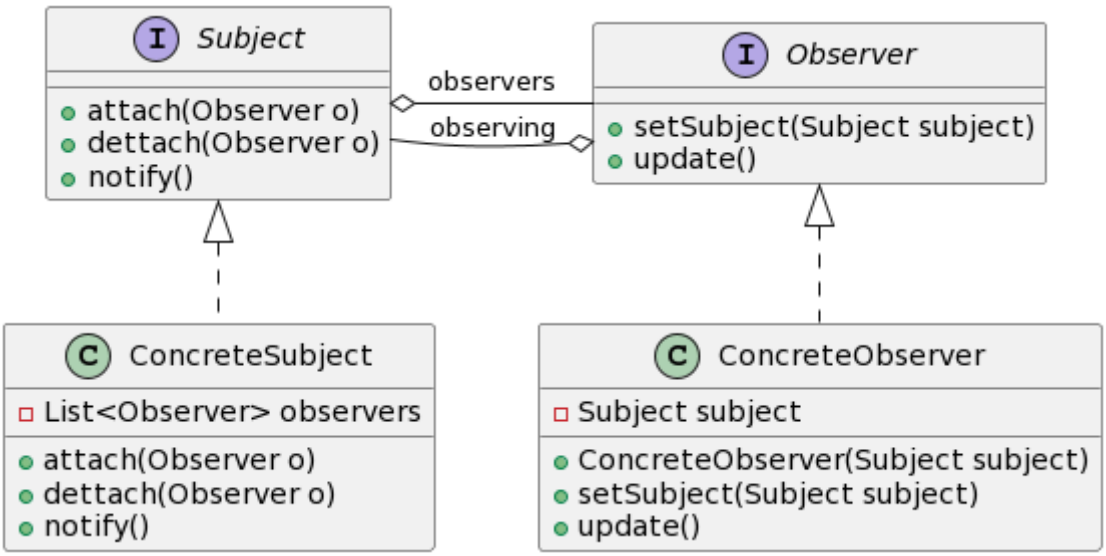
- 发布订阅模式
- 模型-视图模式
- 源-监听器模式
- 从属模式

动机：

- 软件系统：一个对象状态的改变，可能会导致其他对象的状态改变，他们之间将会发生联动关系
- 发生改变的对象叫做**观察目标（Subject）**， 被通知的对象叫做**观察者（Observer）**

定义：定义对象之间的一种**一对多的依赖关系**，使得**每当一个对象的状态发生改变时**，其**相关依赖的对象皆得到通知并且被自动更新**

类图：



分析：

- 有时候在**具体的观察者类中需要使用到具体的目标类中的状态，会存在关联关系**
- 如果在**具体层存在关联关系**，那么系统的扩展性将会受到一定影响，违背**开闭原则。**如果**原有的观察者类无需关联新增的具体目标**，则系统扩展性不受影响。

优点：

- **实现表现层和数据层的分离**
- 目标类和观察者之间建立一个**抽象的耦合**
- 支持**广播通信**
- 符合**开闭原则**

缺点：

- 将所有的观察者**通知**到会**花费很多时间**
- 如果存在**循环依赖**将会导致系统奔溃
- **没有相应的机制让观察者观察到所观察的目标的变化过程**

场景：

- 抽象模型有两方面，**一个方面依赖于另一个方面**
- **一个对象的改变将导致或者多个其他对象发生改变**，并且不知道有多少对象会改变，也不知道这些对象的具体目标

- **需要创建一个触发链**

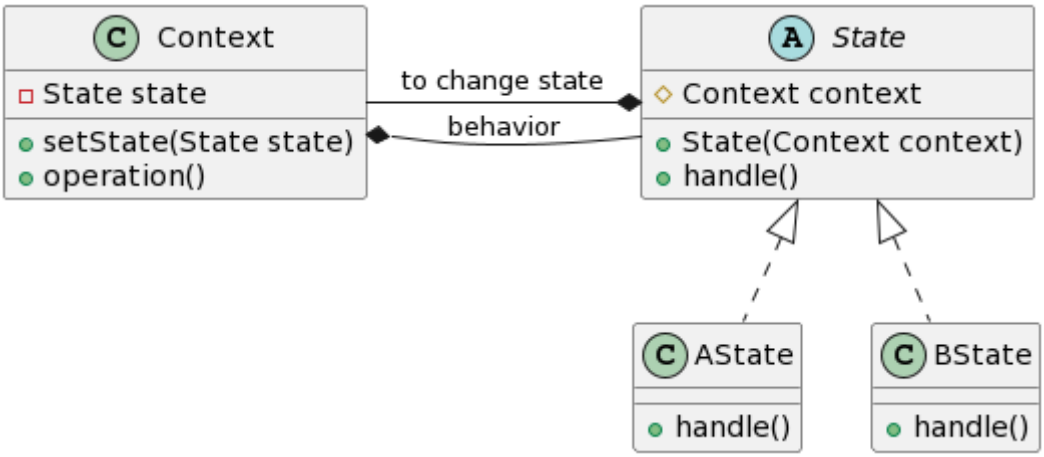
状态模式

动机：

- 软件系统中
  - 有些对象**有很多的状态**
  - 这些状态在**一定条件下能够转换**
  - 对象**在不同的状态下有不同的行为**

定义：允许一个对象在其**内部状态改变时改变其行为，在外界看来似乎是改变了它的类**

类图：



分析：

- **复杂对象的状态转换以及不同状态下的封装问题**
- 将一个对象的**状态**从对象中**分离**出来，**封装**到专门的状态类中
- 客户端无需关心对象状态的转换，以及对象当前所处的状态

优点：

- **封装了状态转换规则**，可以对状态转换代码**集中管理**
- 将所有与**某个状态有关的行为封装**到一个类中
- 允许**将状态转换的逻辑和对象合成一体**，而不是提供一个巨大的条件语句块
- 多个环境对象**共享一个状态对象**，从而**减少系统中对象的个数**

缺点：

- **增加系统中对象的个数**
- 结构和实现较为复杂，**如果使用不当很容易导致程序结构和代码混乱，增加设计难度**
- **对开闭原则支持差**，状态之间依赖于具体的实现，如果状态规则修改，也需要修改对应的源代码

场景：

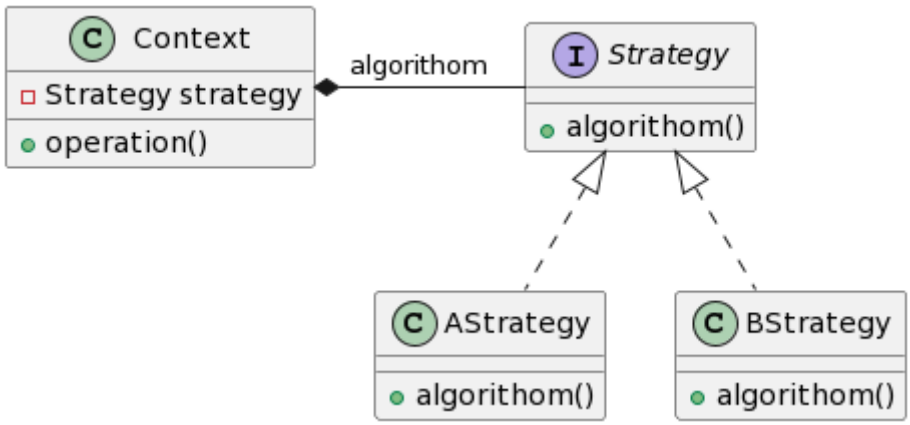
- 对象的行为依赖于状态，状态的改变导致行为的改变
- 在代码中大量包含与状态有关的条件语句

策略模式

动机：实现目标的途径很多的时候，就需要**根据实际情况选择更加合适的途径。**使用**硬编码**实现将导致系统**违背开闭原则，扩展困难，维护困难。**

定义：定义一系列算法，**将每一个算法封装到策略类中，并且让他们可以相互替换。**

类图：



分析：

- 算法可以独立于使用它的客户而变化
- 策略模式提供了一种**可插入式（Pluggable）算法**的实现方案

优点：

- **对开闭原则的完美支持**
- 提供了**管理相关算法的方案**
- 提供了一种可以**替换继承关系的方案**
- 可以**避免多重条件选择**
- 提供了一种**算法的复用机制**，在不同的环境类可以方便地复用策略类

缺点：

- **客户端必须知道所有的策略类**
- 将造成系统**产生大量策略类**
- **无法在客户端同时使用多个策略类**

情景：

- **动态选择算法中的一种**
- **避免难以维护的多重条件选择语句**
- **提高算法的保密性和安全性**